

UNIVERSIDAD INTERNACIONAL DEL ECUADOR

Programación Orientada a Objetos 2



LeeLibre

**Desarrollo de un Sistema de Gestión de Libros Electrónicos
Implementación bajo Programación Orientada a Objetos en Go**

Alumno: Cortez Villa Ronny

Docente: Ing. Torres Arévalo Carlos

Etapas 2 — Desarrollo del Software

9 de agosto de 2026

Resumen

El presente documento corresponde a la Etapa 2 del proyecto integrador de la unidad y describe la implementación de LeeLibre, un sistema de gestión de libros electrónicos que permite consultar un catálogo, registrar préstamos digitales, gestionar una cola de reservas y obtener reportes de uso. A diferencia de la etapa anterior, que se limitó a la planeación, esta entrega presenta un sistema ejecutable compuesto por 2.958 líneas de código Go distribuidas en 16 archivos y 8 paquetes internos, sin ninguna dependencia externa. El desarrollo se realizó aplicando los conceptos de la programación orientada a objetos que corresponden a la asignatura: encapsulación a nivel de paquete, métodos con receptores de valor y de puntero, constructores con validación, captadores y modificadores que protegen las invariantes del dominio, manejo de errores mediante valores centinela comparables con `errors.Is`, e interfaces que habilitan el polimorfismo. Sobre ese núcleo se levantaron dos interfaces de usuario independientes: una aplicación de consola y un servidor de servicios web REST con serialización JSON, protegido mediante un middleware de autenticación. Se documentan además las estructuras de datos empleadas, la funcionalidad diferencial del sistema —un recomendador personalizado de lecturas— y las verificaciones realizadas sobre el código, entre ellas la detección y corrección de doce accesos concurrentes conflictivos.

Palabras clave: programación orientada a objetos, Go, encapsulación, interfaces, polimorfismo, servicios web REST, concurrencia.

Introducción

La Etapa 1 de este proyecto entregó una planeación: qué se iba a construir, con qué módulos y bajo qué arquitectura. Esta segunda etapa convierte esa planeación en un sistema que funciona. El cambio no es solamente de volumen, sino de naturaleza: en la planeación las decisiones se pueden enunciar, mientras que en la implementación hay que sostenerlas frente al compilador, frente a los datos reales y frente al comportamiento del programa en ejecución.

El documento se organiza siguiendo ese recorrido. Primero se delimita qué quedó efectivamente implementado y se justifica el cambio de paradigma respecto de la planeación inicial. Después se presenta el marco referencial de la programación orientada a objetos tal como se expresa en Go, que no dispone de clases ni de herencia tradicional y obliga a construir esos conceptos con otros mecanismos. A continuación se describen la arquitectura y cada uno de los módulos, se detallan los conceptos aplicados con fragmentos del código real, y se explican los servicios web y el esquema de seguridad. Finalmente se reportan las verificaciones realizadas y las conclusiones de la etapa.

Conviene señalar desde el inicio una decisión que atraviesa todo el proyecto: no se utilizó ningún paquete de terceros. El sistema completo se compila únicamente con la biblioteca estándar del lenguaje. Las razones se detallan en la sección de paquetes y librerías.

Objetivos

Objetivo general

Implementar un sistema funcional de gestión de libros electrónicos aplicando los conceptos de la programación orientada a objetos en el lenguaje Go, de modo que el resultado sea un programa ejecutable, verificable y documentado.

Objetivos específicos

1. Construir los módulos del dominio —catálogo, usuarios, préstamos, reservas y reportes— aplicando encapsulación a nivel de paquete, constructores con validación y métodos que protejan las invariantes de cada entidad.
2. Implementar el manejo de errores mediante valores centinela comparables con `errors.Is`, evitando la comparación de mensajes de texto.
3. Diseñar al menos una interface con múltiples implementaciones que permita demostrar polimorfismo, y emplear las estructuras de datos vistas en la asignatura donde el problema lo justifique.
4. Exponer el sistema mediante servicios web REST con serialización JSON, incorporando un esquema de autenticación y autorización acorde a los roles definidos en el dominio.

Alcance de la Etapa 2

Lo entregado en esta etapa comprende un sistema operativo con dos formas de uso. La aplicación de consola permite iniciar sesión, consultar y buscar en el catálogo, registrar y devolver préstamos, revisar el historial personal, obtener recomendaciones y consultar la posición en las colas de reserva; en el caso de un usuario administrador, permite además agregar, editar y retirar libros y consultar los reportes del sistema. El servidor de servicios web expone doce rutas que devuelven JSON y cubren las mismas operaciones del dominio.

Quedan fuera de esta etapa, y se abordarán en la siguiente, la interfaz web visual y las pruebas automatizadas. El sistema se ejercitó con un conjunto de datos ficticios compuesto por siete categorías, treinta libros y diez usuarios, cantidad suficiente para que los reportes y las recomendaciones produzcan resultados con sentido.

Del paradigma funcional al orientado a objetos

La planeación de la Etapa 1 propuso desarrollar el sistema bajo el paradigma de programación funcional. Esa orientación se corrigió en esta etapa por una razón de fondo: la asignatura es Programación Orientada a Objetos y su contenido —estructuras, métodos, constructores, encapsulación, manejo de errores e interfaces— pertenece a ese paradigma. Sostener el enfoque funcional habría significado no aplicar los temas que la unidad desarrolla.

El cambio no descarta lo planificado, sino que lo reinterpreta. La idea de operaciones que reciben funciones como parámetro se conservó donde resultaba natural: el catálogo expone un método **Filtrar** que recibe un predicado, y las búsquedas por texto, por categoría y por disponibilidad se construyen sobre él en lugar de repetir el recorrido. Las funciones siguen siendo valores de primera clase en Go, y esa propiedad se aprovecha; lo que cambia es que el estado del sistema se modela con entidades que protegen sus propios datos, en vez de con transformaciones sobre estructuras inmutables.

Marco Referencial

La programación orientada a objetos en Go

Go no tiene clases, ni herencia, ni las palabras reservadas **public** y **private**. Sin embargo, sí permite expresar los conceptos centrales del paradigma mediante otros mecanismos, y comprender esa correspondencia fue parte del trabajo de esta etapa.

El equivalente de la clase es el *struct* acompañado de sus métodos. Un método es una función asociada a un tipo mediante un receptor declarado antes del nombre. El receptor puede ser de valor, en cuyo caso el método opera sobre una copia, o de puntero, en cuyo caso opera sobre el objeto original; esta distinción es determinante, porque un método que modifica el estado y usa receptor de valor no produce ningún efecto observable.

La encapsulación se resuelve por convención de nombres y su ámbito es el paquete: los identificadores que comienzan con minúscula no son visibles fuera de su paquete, y los que

comienzan con mayúscula sí lo son. No existe un nivel intermedio equivalente al `protected` de otros lenguajes.

El polimorfismo se obtiene mediante interfaces con satisfacción implícita: un tipo implementa una interface por el solo hecho de tener los métodos que ésta declara, sin necesidad de anunciarlo. Esta característica, conocida como *duck typing* estructural, permite que un paquete defina una interface y que otros paquetes la satisfagan sin depender de él.

Errores como valores

Go no utiliza excepciones. Una función que puede fallar devuelve un error como último valor de retorno, y el llamador debe atenderlo de forma explícita. Para poder distinguir la causa de un fallo sin recurrir a comparar mensajes de texto, el proyecto define errores centinela en un paquete propio y los envuelve con el verbo `%w` cuando necesita añadir contexto, de modo que la comparación posterior mediante `errors.Is` siga funcionando.

Concurrencia

El servidor web de la biblioteca estándar atiende cada petición en una *goroutine* independiente. Esto implica que varias peticiones pueden ejecutarse simultáneamente sobre las mismas estructuras de datos. Los mapas y los segmentos de Go no son seguros para uso concurrente, de manera que cualquier estado compartido debe protegerse explícitamente. El proyecto utiliza `sync.RWMutex`, que distingue entre accesos de lectura, que pueden ser simultáneos, y accesos de escritura, que son exclusivos.

Arquitectura del Sistema

La arquitectura mantiene la separación por capas anticipada en la planeación. La diferencia respecto de lo previsto es que la capa de presentación se materializó en dos programas distintos que comparten el mismo núcleo de dominio, en lugar de en una sola aplicación web.

```
|-- cmd/
|   |-- leelibre/main.go    # aplicación de consola
|   |-- server/main.go     # servidor de servicios web
|-- internal/
|   |-- errores/           # errores centinela del dominio
|   |-- catalogo/         # libros, categorías y catálogo
|   |-- usuarios/         # usuarios, roles y autenticación
|   |-- prestamos/        # préstamos e historial
|   |-- reservas/         # cola FIFO de espera
|   |-- reportes/         # interface Reportador e implementaciones
|   |-- persistencia/     # escritura de los datos en JSON
|   |-- api/              # controladores HTTP y middleware
|-- data/                 # datos de ejemplo en JSON
|-- docs/                 # documentación de las etapas
```

La dirección de las dependencias es un criterio que se sostuvo durante todo el desarrollo: los paquetes del dominio no conocen a la capa de presentación, y el paquete `catalogo`, que es el más básico, no depende de `usuarios` ni de `prestamos`. El paquete `reportes` es el único que consulta a varios, lo cual es coherente con su función de agregar información. Gracias a esta disposición, los dos ejecutables consumen los mismos paquetes internos y la lógica de negocio no se encuentra duplicada.

Descripción de los Módulos Implementados

Módulo de errores

Concentra los errores del dominio como valores exportados creados con `errors.New`. Al estar declarados en un único lugar, cualquier paquete puede compararlos con `errors.Is` sin depender del texto del mensaje. Es el único paquete del que dependen todos los demás y no depende de ninguno.

Módulo de catálogo

Contiene los tipos **Libro**, **Categoria** y **Catalogo**. El libro guarda su título, autor, categoría, año, formato y disponibilidad, todos en campos privados. El catálogo administra la colección y garantiza dos invariantes: que no existan dos libros con el mismo identificador y que ningún libro pertenezca a una categoría inexistente.

Su método **Filtrar** recibe una función que actúa como predicado y devuelve los libros que la satisfacen. Las búsquedas por texto, por categoría y por disponibilidad se implementan sobre él, de modo que la lógica del recorrido está escrita una sola vez.

Módulo de usuarios y autenticación

Define el tipo **Ro1** con dos valores válidos, lector y administrador, y el tipo **Usuario**. La contraseña merece una mención: es el único campo que no tiene captador, de manera que no puede leerse desde fuera del paquete; solo puede comprobarse mediante el método **VerificarPassword**. El autenticador mantiene dos índices, por identificador y por correo, y devuelve el mismo error tanto si el correo no existe como si la contraseña es incorrecta, para no revelar qué correos están registrados.

Módulo de préstamos

Modela el préstamo como una entidad con estado propio: vigente, devuelto o vencido. El préstamo se otorga por catorce días y calcula su fecha de vencimiento en el momento de crearse. El historial administra la colección completa y ofrece consultas por usuario y por libro, además del conteo de préstamos por libro que alimenta el reporte de los más solicitados.

Módulo de reservas

Implementa la rama del diagrama de flujo de la Etapa 1 que contemplaba el caso de un libro no disponible. En lugar de limitarse a informar del rechazo, el sistema ofrece al usuario incorporarse a una cola de espera. La cola es una estructura FIFO con las operaciones

clásicas de encolar, desencolar y consultar el frente; internamente se apoya en un segmento, pero su interfaz pública es la de una cola pura. El gestor mantiene una cola por libro en un mapa indexado por identificador.

Módulo de reportes

Declara la interface `Reportador` y dos implementaciones, descritas más adelante en la sección de polimorfismo.

Módulo de persistencia

Escribe el estado del sistema en archivos JSON. La escritura se realiza de forma atómica: primero se genera un archivo temporal y después se renombra, de modo que una interrupción a mitad del proceso no deje el archivo corrupto.

Módulo de servicios web

Contiene los controladores HTTP, los objetos de transferencia que definen la forma del JSON de salida y el middleware de autenticación. Los objetos de transferencia son propios de esta capa, lo que evita exponer los tipos del dominio y evita también tener que contaminarlos con etiquetas de serialización.

Conceptos de Programación Orientada a Objetos Aplicados

Encapsulación y captadores

Todos los campos de las entidades son privados. El acceso se realiza mediante captadores que en Go no llevan el prefijo `Get`, por convención del lenguaje: el captador del campo `titulo` se llama `Titulo`.

Modificadores con validación de invariantes

Los modificadores son la justificación práctica de que los campos sean privados, porque permiten rechazar un valor que dejaría al objeto en un estado inválido. Aplican las mismas reglas que el constructor, de modo que un libro creado y uno modificado cumplen siempre las mismas condiciones.

```
// SetAnio cambia el año de publicación, dentro del mismo rango que exige el
// constructor.
func (l *Libro) SetAnio(nuevo int) error {
    if nuevo < 1000 || nuevo > 2100 {
        return fmt.Errorf("%w: el año debe estar entre 1000 y 2100, se recibió %d",
            errores.ErrDatosInvalidos, nuevo)
    }
    l.mu.Lock()
    defer l.mu.Unlock()
    l.anio = nuevo
    return nil
}
```

Si los campos fueran públicos, cualquier fragmento del programa podría asignar un año fuera de rango y el libro quedaría en un estado imposible sin que nada lo advirtiera.

Constructores

Cada entidad se crea mediante una función constructora que valida los datos antes de devolver el objeto. Como la construcción puede fallar, el constructor devuelve el objeto y un error, patrón habitual en Go. Devuelve además un puntero, para no crear una copia innecesaria del valor.

Receptores de valor y de puntero

Los métodos que modifican el estado utilizan receptor de puntero. Los que solo consultan podrían usar receptor de valor, pero se emplea puntero por coherencia y para evitar

copias.

Interfaces y polimorfismo

El paquete de reportes declara la interface `Reportador`, que exige dos métodos. La implementan dos tipos distintos: `MasPrestados`, que cuenta apariciones en el historial mediante un mapa y ordena el resultado, y `Recomendador`, que construye el perfil del lector y puntúa los libros que aún no ha leído. Ambos se utilizan de manera uniforme desde la consola y desde los servicios web, sin que el código llamador conozca el tipo concreto.

Un segundo caso de polimorfismo, más sutil, aparece en la interface `fmt.Stringer` de la biblioteca estándar: al definir un método `String` en los tipos `Libro` y `Prestamo`, cualquier función de impresión los muestra ya formateados, sin declaración explícita de que se implementa la interface.

Manejo de errores

Los errores del dominio se comparan con `errors.Is` y se envuelven con `%w` cuando se les añade contexto. El siguiente fragmento, tomado de la aplicación de consola, distingue un error previsible de uno inesperado:

```
if err != nil {
    if errors.Is(err, errdom.ErrDatosInvalidos) {
        fmt.Printf("Datos inválidos: %v\n", err)
        return
    }
    fmt.Printf("Error inesperado: %v\n", err)
    return
}
```

Estructuras de Datos Empleadas

Tabla 1

Estructuras de datos utilizadas y justificación.

Estructura	Dónde se usa	Por qué
Segmento	Catálogo de libros, historial de préstamos	La colección crece y se recorre completa con frecuencia
Mapa	Índices por identificador y por correo, perfil del lector, conteo por libro	Acceso directo por clave sin recorrer la colección
Cola FIFO	Reservas de un libro no disponible	El orden de llegada determina quién recibe el libro primero

El perfil del lector del recomendador ilustra bien la elección del mapa: se construye recorriendo el historial una sola vez y acumulando cuántas lecturas corresponden a cada categoría, operación que con un segmento exigiría una búsqueda por cada préstamo.

Funcionalidad No Básica: el Recomendador

Además de las operaciones habituales de altas, bajas, consultas y préstamos, el sistema incorpora un recomendador personalizado, que constituye la funcionalidad diferencial de esta entrega.

Su funcionamiento consta de tres pasos. Primero recupera el historial del usuario y construye su perfil de lector, es decir, un mapa que asocia cada categoría con la cantidad de libros leídos de ella. Después recorre el catálogo y descarta los libros que el usuario ya leyó. Por último asigna a cada libro restante una puntuación igual a la proporción de lecturas del usuario que pertenecen a la categoría de ese libro, y ordena los resultados de mayor a menor.

El recomendador no se limita a devolver una lista: acompaña cada sugerencia con el motivo que la justifica. Para una usuaria con nueve lecturas registradas, de las cuales seis son de ficción, la respuesta indica una puntuación de 0,67 y la razón «Coincide con tu interés en Ficción (6 de 9 lecturas)». Esta explicación convierte una sugerencia opaca en una

recomendación comprensible.

Servicios Web REST

El servidor expone doce rutas construidas sobre `net/http`. Todas responden en JSON.

La tabla siguiente resume las rutas y el nivel de acceso requerido.

Tabla 2

Rutas expuestas por el servidor y nivel de acceso.

Método	Ruta	Acceso
GET	/	Público
GET	/api/libros	Público
GET	/api/libros/{id}	Público
POST	/api/libros	Administrador
PUT	/api/libros/{id}	Administrador
DELETE	/api/libros/{id}	Administrador
GET	/api/categorias	Público
POST	/api/prestamos	Autenticado
POST	/api/prestamos/{id}/devolver	Dueño o administrador
GET	/api/usuarios/{id}/recomendaciones	Público
GET	/api/reportes/mas-prestados	Público
GET	/api/reservas/{libro_id}	Público

Autenticación y Autorización

Las operaciones que modifican datos exigen credenciales. El mecanismo elegido es la autenticación básica de HTTP, resuelta con la función `BasicAuth` de la biblioteca estándar, lo que evita inventar un formato propio de credencial.

El control se implementó como *middleware*, es decir, como una función que recibe un controlador y devuelve otro con comportamiento añadido a su alrededor. La función `conAuth` exige credenciales válidas y entrega el usuario ya identificado al controlador; la función `conAdmin` se construye sobre la anterior y añade la comprobación del rol. De este modo la verificación no se repite dentro de cada controlador y resulta imposible olvidarla al añadir uno nuevo.

Esta construcción distingue dos preguntas que suelen confundirse. La primera es quién realiza la petición: si no puede establecerse, la respuesta es 401. La segunda es si quien la realiza tiene permiso: si no lo tiene, la respuesta es 403. Existe además una tercera comprobación, que atiende al objeto concreto sobre el que se actúa: devolver un préstamo exige ser su titular, con el administrador como excepción, porque estar autenticado no basta para operar sobre un recurso ajeno.

Una decisión relacionada afecta al registro de préstamos. En una primera versión el identificador del usuario viajaba en el cuerpo de la petición, lo que permitía solicitar libros a nombre de otra persona. El préstamo se registra ahora a nombre de quien se autentica, y el cuerpo de la petición únicamente indica qué libro se solicita: la identidad del solicitante no puede provenir del cliente.

Concurrencia y Protección del Estado Compartido

Al exponer el sistema como servidor web apareció una condición que la aplicación de consola no presentaba. Dado que cada petición se atiende en una goroutine distinta, dos peticiones simultáneas pueden modificar el catálogo, el historial o las colas al mismo tiempo, y ni los segmentos ni los mapas de Go admiten ese uso.

La verificación con el detector de carreras del lenguaje confirmó el problema: al someter al servidor a peticiones concurrentes se registraron doce accesos conflictivos, localizados en las operaciones de alta de libros, registro de préstamos y reserva.

La corrección consistió en incorporar un `sync.RWMutex` a las colecciones y a las dos entidades cuyo estado cambia después de construirse. Las consultas toman el candado en modo lectura, que admite varios lectores simultáneos, y las modificaciones lo toman en modo escritura, que es exclusivo. Dos detalles del diseño merecen mención. El primero es que la comprobación y la escritura ocurren bajo un mismo bloqueo: si se realizaran por separado, dos peticiones podrían comprobar que un libro está disponible y prestarlo ambas. El segundo es que `sync.RWMutex` no es reentrante, por lo que los métodos que se apoyan en otros del

mismo tipo utilizan una versión interna sin bloqueo, o bien delegan en el único método que lo toma.

El orden de adquisición es siempre el mismo —colección antes que entidad— para que dos goroutines no puedan quedar esperándose mutuamente. Tras la corrección, la misma prueba no registró ningún acceso conflictivo.

Paquetes y Librerías

Tabla 3

Paquetes de la biblioteca estándar utilizados.

Paquete	Uso en el proyecto
net/http	Servidor y enrutamiento de los servicios web
encoding/json	Serialización y lectura de los datos y de las respuestas
errors	Errores centinela y comparación con errors.Is
fmt	Formato de mensajes y envoltura de errores con %w
sync	RWMutex para proteger el estado compartido
time	Fechas de préstamo, vencimiento y devolución
os	Lectura y escritura de los archivos de datos
strings	Normalización de texto en las búsquedas
strconv	Conversión de parámetros de las rutas
sort	Ordenamiento de los reportes
bufio	Lectura de la entrada en la aplicación de consola

La planeación de la Etapa 1 preveía utilizar cuatro paquetes de terceros: un framework web, un controlador de base de datos, una librería de tokens y otra de cifrado. Ninguno se incorporó. La decisión se sostiene en tres razones. La primera es que la biblioteca estándar de Go cubre las necesidades del proyecto: desde la versión 1.22 su enrutador admite métodos y variables de ruta, que era la principal comodidad que aportaba un framework. La segunda es que el proyecto compila sin descargar nada, lo que simplifica su ejecución y su evaluación. La tercera es que el objetivo de la asignatura es demostrar dominio del lenguaje, y apoyarse en un framework habría desplazado a él buena parte de las decisiones de diseño.

En lugar de una base de datos se emplean archivos JSON. Para el volumen de datos

previsto la diferencia de rendimiento es irrelevante, y la capa de persistencia quedó aislada, de modo que una migración futura no afectaría al resto de los módulos.

Verificación Realizada

El código fue sometido a las herramientas de análisis del propio lenguaje. La compilación completa no produce advertencias, el analizador estático `go vet` no reporta observaciones y el formateador `gofmt` no encuentra desviaciones respecto del estilo oficial.

El comportamiento se verificó ejecutando el sistema. Por la vía de los servicios web se comprobó que una petición sin credenciales recibe 401, que un lector que intenta eliminar un libro recibe 403, que un administrador completa la misma operación, y que un usuario que envía el identificador de otro obtiene igualmente el préstamo a su propio nombre. Se comprobó también que los modificadores rechazan un año fuera de rango, un título vacío y un formato inexistente, devolviendo en los tres casos un 400 y conservando el libro sus valores anteriores.

Por la vía de la consola se verificó el recorrido completo del caso principal: una lectora obtiene recomendaciones, registra un préstamo y cierra sesión; otro lector solicita el mismo libro, recibe el aviso de que no está disponible y se incorpora a la cola de reservas en la primera posición.

Finalmente, el análisis de concurrencia descrito anteriormente se ejecutó antes y después de la corrección, con los resultados ya indicados.

La ausencia de pruebas automatizadas es la principal limitación de esta etapa. Su incorporación está prevista para la Etapa 3 y exigirá una modificación puntual: el método que actualiza el estado de un préstamo consulta internamente la fecha del sistema, y para poder probarlo de forma determinista deberá recibir esa fecha como parámetro.

Repositorio y Control de Versiones

El proyecto se encuentra publicado en el repositorio <https://github.com/ronnycort/leelibre>. El historial acumula veinte confirmaciones y se organizó de manera que cada una corresponda a una unidad con sentido propio: un paquete del dominio, la capa de servicios web, o una mejora concreta. El orden respeta las dependencias entre paquetes, de modo que cada confirmación compila por sí sola.

Se mantienen las dos ramas previstas en la planeación, `main` para las versiones estables de cada etapa y `develop` para el trabajo en curso. Los mensajes de las confirmaciones explican la razón del cambio y no solamente su contenido, criterio que resulta útil al revisar el historial tiempo después.

Conclusiones Parciales de la Etapa 2

La etapa cumplió su objetivo: el sistema planificado existe, se ejecuta y puede demostrarse. Del trabajo realizado se desprenden cuatro observaciones.

La primera es que el cambio del paradigma funcional al orientado a objetos, lejos de invalidar la planeación, permitió aprovechar mejor los contenidos de la asignatura. Las ideas funcionales que sí encajaban —como el filtrado mediante un predicado— se conservaron, y el resto del sistema se modeló con entidades que protegen su propio estado.

La segunda es que la encapsulación adquiere sentido práctico cuando aparecen los modificadores. Mientras solo existen captadores, hacer privados los campos parece una formalidad; cuando un modificador rechaza un año imposible y devuelve un error, la razón de ser de esa decisión queda a la vista.

La tercera es que exponer el sistema como servidor web introdujo una clase de problema que no existía en la consola. Ni el compilador ni el analizador estático advierten de un acceso concurrente conflictivo; hizo falta ejecutar el programa bajo el detector de carreras para encontrarlo. La verificación en ejecución no es sustituible por la lectura del código.

La cuarta se refiere a la seguridad: añadir autenticación no resuelve por sí solo el problema de la suplantación. Mientras el identificador del usuario siguió llegando en el cuerpo de la petición, cualquiera podía actuar en nombre de otro pese a estar correctamente autenticado. La identidad de quien realiza una acción debe provenir de la sesión y nunca del contenido que envía el cliente.

Con esta base, la Etapa 3 puede concentrarse en la interfaz web visual, en las pruebas automatizadas y en la documentación final del proyecto.

Referencias

- 1 American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7a ed.). <https://doi.org/10.1037/0000165-000>
- 2 Donovan, A. A. A., & Kernighan, B. W. (2015). *The Go programming language*. Addison-Wesley.
- 3 Sommerville, I. (2016). *Software engineering* (10a ed.). Pearson.
- 4 Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2a ed.). Addison-Wesley.
- 5 The Go Authors. (2024). *Effective Go*. https://go.dev/doc/effective_go
- 6 The Go Authors. (2024). *The Go memory model*. <https://go.dev/ref/mem>
- 7 Fielding, R. T., & Reschke, J. (2014). *Hypertext transfer protocol (HTTP/1.1): Semantics and content* (RFC 7231). Internet Engineering Task Force. <https://doi.org/10.17487/RFC7231>
- 8 Meyer, B. (1997). *Object-oriented software construction* (2a ed.). Prentice Hall.